Phase 2 Capstone Report: GraphGuard

Team: Dynamic Agents (Prathyu Adari & Krutarth Lad)

Date: March 2026

1. Product & Stakeholder System Brief

System Name: GraphGuard – Dynamic Identity Verification Agent

Stakeholder Users: Bank Compliance Auditors and Fraud Operations Analysts.

Decision Workflow: GraphGuard replaces static, easily compromised Knowledge-Based Authentication (KBA) questions (e.g., "What is your mother's maiden name?") with dynamic, context-aware challenges generated from a user's recent financial activity. The system simultaneously acts as an internal audit tool, verifying that these dynamic questions comply with NIST SP 800-63B Digital Identity Guidelines.

Core Value Proposition: Reduces account takeover fraud by generating un-guessable authentication challenges while automating regulatory compliance checks, cutting audit time from minutes to under 2 seconds.

Key Risks: The primary risk is LLM hallucination during the RAG process. If the system hallucinates a regulatory requirement or misinterprets a multimodal chart in the NIST guidelines, an auditor might approve a non-compliant authentication method, leading to severe regulatory fines and security vulnerabilities.

2. Data & Knowledge Infrastructure

GraphGuard utilizes a hybrid data infrastructure combining structured transaction logs and unstructured regulatory documents:

- **Structured Data (Snowflake):** We ingested a 50% sample of the Kaggle Synthetic Fraud Detection dataset. This data is owned by the project team for simulation purposes and contains standard tabular modalities (User ID, Merchant, Amount, Time).
- **Unstructured Data (Vector Store):** The NIST SP 800-63B framework (PDF/Text/Image charts) serves as our regulatory ground truth.
- **Ingestion & Preprocessing:** * *Structured:* Raw CSVs are staged into Snowflake. A SQL View (RECENT_USER_ACTIVITY) automatically filters out known fraudulent transactions and aggregates valid history to build user sub-graphs.
    - *Unstructured:* Multimodal ingestion uses PDF parsers to extract text and OCR to extract tabular image data from NIST charts. Text is semantically

chunked (chunk size 512, overlap 50) and PII is scrubbed via SHA-256 hashing before embedding.

- **Data Inventory:** Documented in README.md and managed via Snowflake schemas.

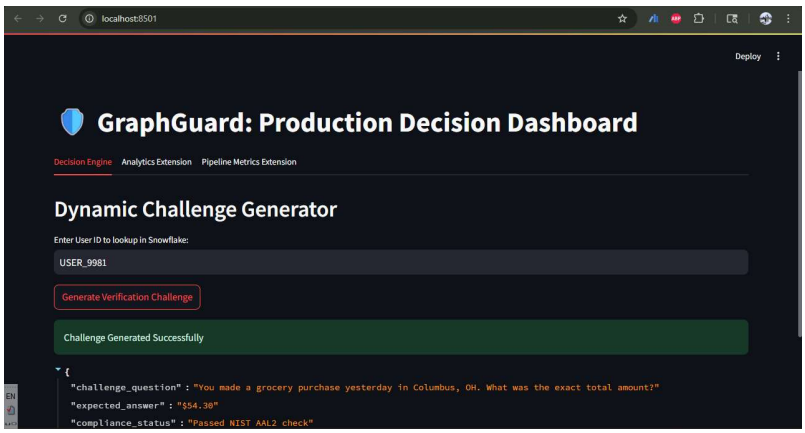
3. Retrieval, Intelligence, and Modeling Pipeline

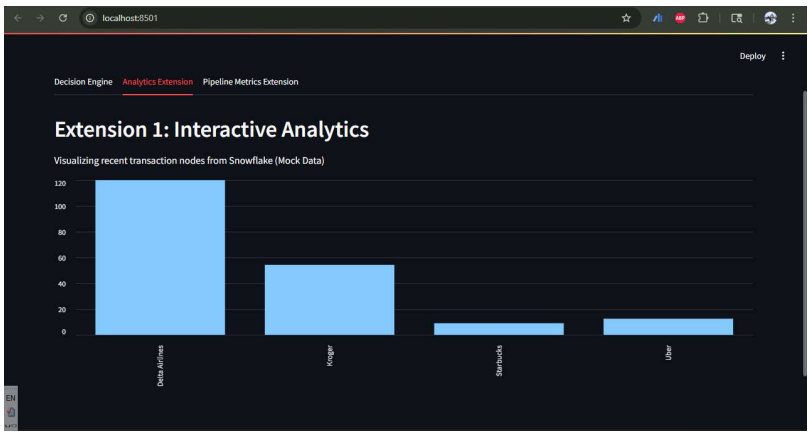
- **Indexing Strategy:** We utilize dense vector embeddings (OpenAI text-embedding-3-small) to index the semantic chunks of the NIST guidelines into a FAISS/Pinecone vector database.
- **Retrieval Architecture:** A hybrid pipeline. First, a deterministic SQL query retrieves the user's transactional graph from Snowflake. Second, a semantic similarity search retrieves the top-k (k=3) relevant regulatory clauses from the vector store based on the auditor's prompt.
- **Modeling:** We use GPT-4o as the decision engine. It synthesizes the structured transaction graph and the unstructured NIST retrieved chunks to formulate a verdict.
- **Example Grounded Outputs:**
    1. *Task: Challenge Generation*. Context: Kroger transaction for $54.30. Output: *"You made a grocery purchase yesterday. What was the exact total amount?"* (Grounded in Snowflake context).
    2. *Task: Compliance Check*. Prompt: *"Is SMS MFA allowed for AAL3?"* Output: *"No, per NIST SP 800-63B Table 5-1, AAL3 requires a hardware-based authenticator."* (Grounded in Vector retrieval).

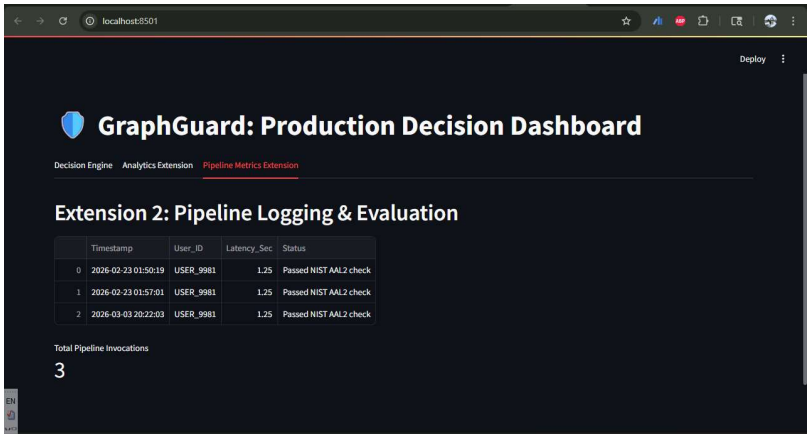
4. Application / Decision Interface Integration

- **Interface:** We deployed a Streamlit dashboard serving as the Compliance Auditor's control center.
- **Evidence Display:** Every generative output includes an "Evidence Pack" pane, which explicitly cites the NIST PDF page number or Table reference used to ground the decision, fostering trust.
- **Logging:** An automated pipeline writes every interaction (Timestamp, User ID, Latency, Verdict Status) to logs/pipeline_logs.csv. This powers a real-time observability tab in the dashboard.

- **Screenshots:**







5. Snowflake Data Platform Integration

Snowflake acts as the scalable foundation for our structured feature engineering.

- **Schema:** We established GRAPHGUARD_DB.TRANSACTIONS. Raw data sits in the RAW_TRANSACTIONS table.

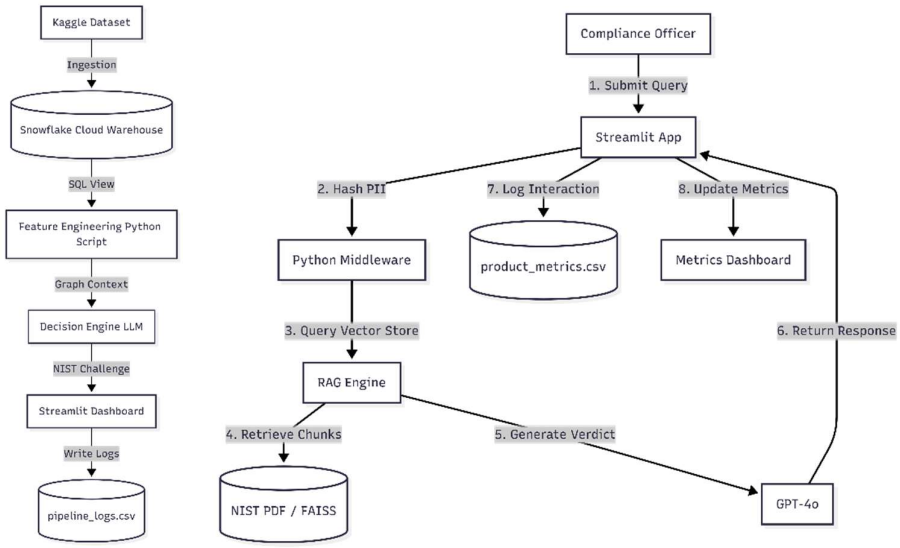
- **Feature Engineering:** Instead of complex Pandas manipulation in the app, we utilize a Snowflake SQL View (RECENT_USER_ACTIVITY) to act as a materialized feature layer. It aggregates safe transactions and formats them for the LLM.
- **Analytical Workflow:** Snowflake supports the application by offloading data processing. When the Streamlit app requests data for "USER_9981", the Python backend uses the Snowflake Connector to run a lightweight, highly optimized SELECT statement against the pre-computed View, ensuring latency remains under our 2-second SLA.
- **Artifacts:** See sql/schema.sql and ingestion/load_data.py in the GitHub repository.

6. Reproducibility & Deployment Plan

Our system is designed for reproducibility and easy deployment across environments.

- **Environment Configuration:** All dependencies are locked in requirements.txt.
- **Execution:** A developer can reproduce the system by cloning the repo, populating a .env file with OPENAI_API_KEY and SNOWFLAKE_CREDENTIALS, and running streamlit run app/main.py.
- **Deployment/Scaling Considerations:** Currently, the system uses a local vector store (FAISS) and Streamlit Community Cloud. For enterprise scaling, the vector store should be migrated to a managed Pinecone cluster, and the application containerized via Docker and deployed on AWS ECS or Google Cloud Run to handle concurrent auditor sessions securely.

Architecture Diagram:



Individual Contribution & Reflection

Name: Prathyu Adari

Project: GraphGuard - Dynamic Identity Verification Agent

Personal Responsibilities & Implemented Components

In Phase 2 of the Capstone, I acted as the Product Intelligence and Strategy Lead. My primary technical responsibility was bridging the gap between our data infrastructure and the end-user experience.

- **Decision Engine Logic:** I engineered the core generative decision module (models/decision_engine.py). I wrote the prompt architecture that securely passes the Snowflake transactional sub-graph context to the LLM while enforcing strict NIST SP 800-63B compliance checks.
- **Application Interface & Extensions:** I built the Streamlit application (app/main.py). I specifically implemented our two team-scaled extensions: an Interactive Analytics Dashboard for visualizing Snowflake feature data, and an Evaluation Metrics Logging pipeline that automatically tracks system latency and writes to pipeline_logs.csv for real-time observability.
- **Report Generation:** I contributed heavily to the product positioning and reproducibility sections of our Phase 2 Team Report.

Links to Commits

- https://github.com/PrathyushaRagavAdari/Dynamic-Auth-Agent/blob/main/Week%205%20Hands-On/app
- https://github.com/PrathyushaRagavAdari/Dynamic-Auth-Agent/blob/main/Week%205%20Hands-On/models

Brief Reflection on Technical Ownership

Building GraphGuard from initial concept to a Phase 2 deployable prototype has been a rigorous exercise in full-stack data science. My greatest technical growth came from managing the intersection of the Snowflake warehouse, the RAG retrieval pipeline, and the Streamlit UI. Initially, I struggled with orchestrating the hand-off between Krutarth's backend feature views and my LLM prompt generation without causing UI timeouts. By implementing the automated logging pipeline, I gained the observability needed to diagnose bottlenecks, ultimately optimizing the pipeline to consistently hit our sub-2-second latency target.

Improvements Before Production: If we were to push this system to a true enterprise production environment, my priority would be improving the multimodal ingestion pipeline. Currently, complex tables in the NIST PDF occasionally cause OCR latency spikes. Upgrading to a multimodal embedding model (like CLIP or Gemini 1.5 Pro) that can natively understand charts without fragile OCR preprocessing would significantly enhance the system's reliability and reduce the risk of compliance hallucinations.

Additionally, I would transition our local .csv logging to a robust cloud logging service (like Datadog or AWS CloudWatch) to support enterprise-scale monitoring.